

项目说明

校园电竞社团管理系统

项目简介

本系统是一个基于 Java 的命令行应用程序，用于管理校园电竞社团的 LOL 和王者荣耀两类游戏赛事。系统实现了选手管理、战队管理、赛事创建、积分统计和选手黑名单管控等基础功能。

技术栈

- 语言:** Java 8+
- 运行方式:** 命令行控制台程序
- 数据存储:** 内存存储 (ArrayList、HashMap、HashSet)

文件结构

```
train01-0709/
├── Operate.java          # 管理员操作接口
├── AbstractPlayer.java   # 选手抽象父类
├── Gamer.java            # 参赛选手子类
├── MatchAdmin.java       # 赛事管理员子类
├── Team.java             # 战队实体类
├── MatchRecord.java      # 对局记录实体类
├── Main.java             # 主程序入口
├── bin/                  # 编译输出目录
└── 项目说明.md           # 项目说明文档
```

类结构设计

1. 接口与抽象类

类/接口	类型	说明
Operate	接口	定义管理员标准业务行为：创建对局、统计积分、拉黑选手
AbstractPlayer	抽象类	统一抽取编号、姓名、年龄、黑名单状态，定义抽象方法 showInfo()

2. 子类

类	继承/实现	独有属性	功能
Gamer	继承 AbstractPlayer	擅长游戏、所属战队ID	查看信息、加入/退出战队、查询参赛记录
MatchAdmin	继承 AbstractPlayer + 实现 Operate	无	创建对局、统计积分、查询全部记录、拉黑选手

3. 实体类

类	属性	规则
Team	编号、名称、游戏、积分、队员集合	单队最多5人
MatchRecord	编号、对战双方、双方积分、比赛结果	胜利+3分，平局各+1分

功能模块

选手功能

1. **查看个人信息**：显示基础身份信息 + 擅长游戏 + 所属战队
2. **加入战队**：三层校验（黑名单拦截、已有战队拦截、战队满员拦截）
3. **退出战队**：清空自身战队标识
4. **查询个人参赛记录**：筛选本人所属战队参与的全部比赛

管理员功能

1. **创建比赛对局**：输入两支战队ID和比赛结果，自动更新积分并生成记录
2. **按游戏分类统计积分**：遍历输出每款游戏下所有战队及总积分
3. **查询系统全部对局记录**：展示所有比赛记录
4. **拉黑/解除拉黑选手**：修改选手黑名单状态

数据容器

容器	类型	用途
teams	ArrayList	存储全部战队
players	ArrayList	存储全部玩家（选手+管理员）
matchRecords	HashMap	存储所有对局记录（key为对局编号）
gameTypes	HashSet	自动去重存储游戏分类

预置测试数据

战队

ID	名称	游戏	初始积分
1	哈基米南北路多	王者荣耀	3
2	大狗叫	LOL	0
3	叮咚鸡	王者荣耀	0

| 选手

ID	姓名	年龄	擅长游戏	所属战队
101	cy	20	王者荣耀	哈基米南北路多(1)
102	qzh	19	LOL	无
103	xzh	21	王者荣耀	叮咚鸡(3)

| 管理员

ID	姓名	年龄
999	管理员	25

| 预置对局记录

对局ID	对战双方	积分	结果
1	哈基米南北路多 VS 叮咚鸡	3:0	A胜

| 运行方式

```
# 进入项目目录
cd train01-0709

# 编译到bin目录（需指定UTF-8编码）
javac -encoding UTF-8 -d bin *.java

# 运行
java -cp bin Main
```

| 使用说明

| 主菜单

校园电竞社团管理系统

请选择登录角色：

1. 参赛选手登录
2. 赛事管理员登录
0. 退出系统

请选择：

| 选手登录

输入选手编号（101/102/103）进入选手菜单：

选手功能菜单

1. 查看个人信息
2. 加入战队
3. 退出战队
4. 查询个人参赛记录
0. 返回主菜单

请选择操作：

| 管理员登录

输入编号999进入管理员菜单：

管理员功能菜单

1. 创建比赛对局
2. 按游戏分类统计积分
3. 查询系统全部对局记录
4. 拉黑/解除拉黑选手
0. 返回主菜单

请选择操作：

| 积分规则

- **胜利战队**：+3 积分
- **平局**：两队各 +1 积分
- **失败战队**：无积分

| 战队人数限制

单支战队最多容纳5名选手，满员后无法新增选手。

| 黑名单规则

- 被拉黑的选手无法加入战队
- 被拉黑的选手无法参与赛事

- 管理员可随时解除拉黑状态

代码逐行详解

1. Operate.java（管理员操作接口）

行号	代码	解释
1	<code>/*定义公共接口 Operate */</code>	注释：说明接口用途
2	<code>public interface Operate {</code>	定义公共接口 Operate，接口名首字母大写，使用 interface 关键字
3	<code>/*创建比赛对局，参数为两队ID和比赛结果，无返回值 */</code>	注释：说明方法功能
4	<code>void createMatch(int teamAId, int teamBId, String result);</code>	声明抽象方法：创建比赛对局
5	<code>/*按游戏分类统计积分，无参数无返回值 */</code>	注释：说明方法功能
6	<code>void statScoreByGame();</code>	声明抽象方法：按游戏分类统计积分
7	<code>/*声明抽象方法：拉黑选手，参数为选手ID，无返回值 */</code>	注释：说明方法功能
8	<code>void blacklistPlayer(int playerId);</code>	声明抽象方法：拉黑选手
9	<code>}</code>	接口定义结束

接口作用：规范管理员必须实现的三个操作方法，实现接口的类必须重写所有抽象方法。

2. AbstractPlayer.java（选手抽象父类）

行号	代码	解释
1-6	<code>/*抽象父类AbstractPlayer...代码复用 */</code>	类级注释：说明抽象类的设计意图
8	<code>public abstract class AbstractPlayer {</code>	定义公共抽象类，使用 abstract 关键字，不能直接实例化
10-11	<code>/*protected 是面向对象编程中的访问修饰符...阻止外部无关代码访问。 */</code>	注释：解释 protected 关键字作用
12	<code>protected int id;</code>	声明 protected 修饰的编号字段，子类可直接访问
13	<code>protected String name;</code>	声明 protected 修饰的姓名字段
14	<code>protected int age;</code>	声明 protected 修饰的年龄字段

行号	代码	解释
15	<code>protected boolean banned;</code>	声明 <code>protected</code> 修饰的黑名单状态字段， <code>true</code> 表示已拉黑
17	<code>public AbstractPlayer(int id, String name, int age) {</code>	构造方法：接收编号、姓名、年龄三个参数
18-21	<code>this.id = id; ... this.banned = false;</code>	将参数赋值给成员变量，初始化黑名单为 <code>false</code>
22	<code>}</code>	构造方法结束
24	<code>public abstract void showInfo();</code>	声明抽象方法 <code>showInfo()</code> ，子类必须重写
26	<code>/*提供公共方法打印基础身份信息，代码复用 */</code>	注释：说明方法作用
27	<code>public void printBasicInfo() {</code>	定义公共方法：打印基础身份信息
28-31	<code>System.out.println("编号: " + id); ...</code>	输出编号、姓名、年龄、黑名单状态
31	<code>System.out.println("黑名单状态: " + (banned ? "禁赛" : "正常"));</code>	三目运算符：禁赛或正常
32	<code>}</code>	方法结束
34	<code>/*获取的编号、姓名、年龄、是否禁赛黑名单getter方法 */</code>	注释：说明以下是getter方法
36-50	<code>public int getId() { return id; } ...</code>	<code>getId()</code> 、 <code>getName()</code> 、 <code>isBanned()</code> 、 <code>setBanned()</code> 方法
51	<code>}</code>	类定义结束

抽象类作用：统一抽取选手和管理员的公共属性和方法，实现代码复用，通过抽象方法强制子类实现自己的信息展示逻辑。

3. Team.java（战队实体类）

行号	代码	解释
1	<code>import java.util.ArrayList;</code>	导入 <code>ArrayList</code> 集合类
3	<code>/*存储队员列表 */</code>	注释：说明导入用途
5-7	<code>/*战队Team...满员后无法新增选手。 */</code>	类级注释：说明战队属性和规则
8	<code>public class Team {</code>	定义公共类 <code>Team</code>
10-14	<code>private int id; ... private ArrayList<Gamer> players;</code>	声明私有字段：编号、名称、游戏、积分、队员集合
16	<code>public Team(int id, String name, String game) {</code>	构造方法

行号	代码	解释
17-21	<code>this.id = id; ... this.players = new ArrayList<>();</code>	赋值所有字段，初始化积分为0
22	<code>}</code>	构造方法结束
24	<code>/*获取战队编号、战队名称...getter方法 */</code>	注释：说明以下是getter方法
26-44	<code>public int getId() { return id; } ...</code>	各字段的getter方法
46	<code>public void addScore(int points) {</code>	添加积分的方法
47	<code>score += points;</code>	将points累加到score上
48	<code>}</code>	方法结束
50	<code>/* 添加队员的方法*/</code>	注释：说明方法作用
51	<code>public boolean addPlayer(Gamer gamer) {</code>	添加队员的方法，返回是否添加成功
52-57	检查人数上限5人，满员返回false，否则添加并返回true	
59	<code>public void showTeamInfo() {</code>	显示战队详细信息的方法
60-64	输出战队ID、名称、游戏、积分、人数	
65	<code>System.out.println("队员列表:");</code>	输出队员列表标题
66	<code>/*遍历队员集合中的每个Gamer对象 */</code>	注释：说明循环作用
67-71	<code>for (Gamer player : players) { ... }</code>	增强for循环遍历队员并输出
72	<code>}</code>	方法结束
73	<code>}</code>	类定义结束

4. MatchRecord.java（对局记录实体类）

行号	代码	解释
1-3	<code>/*对局记录 MatchRecord...失败无积分。*/</code>	类级注释：说明属性和积分规则
5	<code>public class MatchRecord {</code>	定义公共类MatchRecord
7-12	<code>private int id; ... private String result;</code>	声明私有字段：对局编号、对战双方ID、积分、比赛结果
14-27	构造方法：接收6个参数并赋值给成员变量	
28	<code>}</code>	构造方法结束

行号	代码	解释
30-40	<pre>public int getId() { return id; } ...</pre>	各字段的getter方法
42	<pre>public void showRecord() {</pre>	显示对局记录详情的方法
43-44	<pre>String teamAName = "还没有设置/找到 喵"; ...</pre>	初始化战队名称为可爱提示语
46	<pre>/*遍历Main类中的全局战队集合,根据 ID 找 name*/</pre>	注释: 说明循环目的
47-54	<pre>for (Team team : Main.teams) { ... }</pre>	遍历战队集合, 根据ID查找战队名称
55-67	<pre>System.out.println("对局ID: " + id);</pre>	输出对局ID、对战双方及积分、比赛结果
68	<pre>}</pre>	方法结束
69	<pre>}</pre>	类定义结束

5. Gamer.java (参赛选手子类)

行号	代码	解释
1-3	<pre>/*Gamer参赛选手...查看本人参赛对局。 */</pre>	类级注释: 说明选手属性和功能
5	<pre>/*定义公共类Gamer, 继承AbstractPlayer抽象类 */</pre>	注释: 说明继承关系
6	<pre>public class Gamer extends AbstractPlayer {</pre>	定义公共类Gamer, 继承AbstractPlayer抽象类
8	<pre>/*独有属性: 擅长游戏、所属战队 ID */</pre>	注释: 说明独有属性
9	<pre>private String skillGame;</pre>	独有属性: 擅长的游戏
10	<pre>private int teamId;</pre>	独有属性: 所属战队ID, -1表示无战队
12	<pre>public Gamer(int id, String name, int age, String skillGame) {</pre>	构造方法
13	<pre>/*调用父类的构造方法, 传递id、name、age*/</pre>	注释: 说明super用途
14-15	<pre>/*super 是面向对象编程中的关键字...如何明确指 代"上一层"。*/</pre>	注释: 详细解释super关键字
16	<pre>super(id, name, age);</pre>	调用父类的构造方法
17	<pre>this.skillGame = skillGame;</pre>	赋值擅长游戏
18	<pre>this.teamId = -1;</pre>	初始化战队ID为-1 (无战队)
19	<pre>}</pre>	构造方法结束

行号	代码	解释
21	<code>@Override /*注解：表示这是重写父类的方法 */</code>	注解说明
22	<code>public void showInfo() {</code>	重写抽象方法showInfo()
23	<code>System.out.println("参赛选手信息:");</code>	输出标题（简化格式）
24	<code>printBasicInfo(); /*调用父类的方法打印基础信息（代码复用） */</code>	调用父类方法
25-26	<code>System.out.println("擅长游戏: " + skillGame); ...</code>	输出擅长游戏和所属战队ID
27	<code>}</code>	方法结束
29	<code>public void joinTeam(int teamId) {</code>	加入战队的方法
30	<code>/*检查是否被拉黑 */</code>	注释：第一层校验
31-34	<code>if (banned) { ... return; }</code>	黑名单校验：被拉黑则拒绝加入
36	<code>/*检查是否已有战队 */</code>	注释：第二层校验
37-40	<code>if (this.teamId != -1) { ... return; }</code>	已有战队校验：已有战队则拒绝加入
42	<code>/*声明目标战队变量，初始为null */</code>	注释：查找目标战队
43-55	<code>Team targetTeam = null; ...</code>	遍历全局战队集合查找目标战队
56-59	<code>if (targetTeam.getPlayerCount() >= 5) { ... return; }</code>	第三层校验：战队满员则拒绝加入
60	<code>this.teamId = teamId;</code>	通过所有校验，设置战队ID
61	<code>targetTeam.addPlayer(this);</code>	将自己加入目标战队
62	<code>System.out.println("成功加入战队 " + targetTeam.getName() + "!!");</code>	输出成功信息
63	<code>}</code>	方法结束
65	<code>public void quitTeam() {</code>	退出战队的方法
66-72	判断是否有战队，有则清空战队ID为-1	
74	<code>public void queryPersonalMatches() {</code>	查询个人参赛记录的方法
75-78	判断是否加入战队，未加入则提示无记录	
79	<code>System.out.println("我的参赛记录:");</code>	输出标题（简化格式）
80-91	遍历对局记录，筛选本人战队参与的比赛并显示	
94-100	<code>public int getTeamId() { return teamId; } ...</code>	getter方法
101	<code>}</code>	类定义结束

6. MatchAdmin.java (赛事管理员子类)

行号	代码	解释
1-2	<pre>/*5.MatchAdmin赛事管理员...查看系统全部对局记录。*/</pre>	类级注释：说明管理员功能
4	<pre>/*继承AbstractPlayer并实现Operate接口，必须重写所有接口方法 */</pre>	注释：说明继承和实现关系
5	<pre>public class MatchAdmin extends AbstractPlayer implements Operate {</pre>	继承AbstractPlayer并实现Operate接口
7-9	构造方法：调用父类构造方法	
11	<pre>@Override</pre>	重写父类抽象方法
12	<pre>public void showInfo() {</pre>	实现showInfo()方法
13	<pre>System.out.println("赛事管理员信息:");</pre>	输出标题（简化格式）
14-15	调用父类方法打印基础信息，输出身份标识	
16	<pre>}</pre>	方法结束
18	<pre>@Override</pre>	重写接口方法
20	<pre>/*创建比赛对局的方法 */</pre>	注释：说明方法功能
21	<pre>public void createMatch(int teamAId, int teamBId, String result) {</pre>	创建比赛对局的方法
22-31	<pre>Team teamA = null; ...</pre>	遍历全局战队集合查找两支战队
33-44	校验战队A、B是否存在，校验不能同一战队对战	
46-63	根据比赛结果(A/B/P)计算积分，无效输入则提示	
65-78	更新战队积分，创建对局记录并存入HashMap	
81	<pre>@Override</pre>	重写接口方法
82	<pre>public void statScoreByGame() {</pre>	按游戏分类统计积分
83	<pre>System.out.println("按游戏分类统计积分:");</pre>	输出标题（简化格式）
84-99	遍历游戏类型，对每个游戏下的战队统计积分	
102	<pre>@Override</pre>	重写接口方法
103	<pre>public void blacklistPlayer(int playerId) {</pre>	拉黑选手的方法
104-114	遍历玩家集合查找选手，不存在则提示	
115-118	<pre>if (player instanceof MatchAdmin) { ... }</pre>	校验不能拉黑管理员
119	<pre>player.setBanned(!player.isBanned());</pre>	切换黑名单状态（取反操作）
120-125	输出操作结果	

行号	代码	解释
126	}	方法结束
128	public void queryAllMatches() {	查询全部对局记录（接口外的额外功能）
129	System.out.println("系统全部对局记录");	输出标题（简化格式）
130-137	判断是否为空，不为空则遍历显示所有记录	
138	}	类定义结束

7. Main.java（主程序入口）

行号	代码	解释
1-4	import java.util.ArrayList; ... import java.util.Scanner;	导入所需的集合类和Scanner
6	public class Main {	定义公共类Main
8	/*全局静态变量：存储所有战队，static修饰表示所有对象共享 */	注释：说明静态变量作用
9	public static ArrayList<Team> teams = new ArrayList<>();	全局静态变量：存储所有战队
10-11	public static ArrayList<AbstractPlayer> players = new ArrayList<>();	全局静态变量：存储所有玩家
12-15	public static HashMap<Integer, MatchRecord> matchRecords = new HashMap<>();	全局静态变量：存储所有对局记录
17	public static void initData() {	初始化测试数据的静态方法
18-23	创建三支战队（哈基米南北路多、大狗叫、叮咚鸡）并加入集合	
25-30	创建三名选手（cy、qzh、xzh）并加入集合	
32-33	cy加入战队1，xzh加入战队3	
35-36	创建管理员并加入玩家集合	
38-39	添加游戏类型（王者荣耀、LOL）到HashSet	
41-43	创建预置对局记录，存入HashMap，战队1获得3分	
45-48	输出初始化完成信息和预置数据说明	
49	}	初始化方法结束
51	public static void gamerMenu(Gamer gamer) {	选手功能菜单方法
52	Scanner scanner = new Scanner(System.in);	创建Scanner对象

行号	代码	解释
53	<code>while (true) {</code>	无限循环
54	<code>System.out.println("\n选手功能菜单");</code>	输出菜单标题（简化格式）
55-61	输出菜单选项，读取用户输入	
63-82	switch语句：根据选择执行查看信息、加入战队、退出战队、查询记录等操作	
83	<code>}</code>	while循环结束
84	<code>}</code>	方法结束
86	<code>public static void adminMenu(MatchAdmin admin) {</code>	管理员功能菜单方法
87-124	结构与选手菜单类似，执行创建对局、统计积分、查询记录、拉黑选手等操作	
127	<code>public static void main(String[] args) {</code>	主方法，程序入口
128	<code>Scanner scanner = new Scanner(System.in);</code>	创建Scanner对象
130	<code>initData();</code>	调用初始化方法
132	<code>while (true) {</code>	主菜单无限循环
133	<code>System.out.println("\n校园电竞社团管理系统");</code>	输出系统标题（简化格式）
134-139	输出主菜单选项，读取角色选择	
141-144	如果选择0，输出再见信息并退出循环	
146-147	提示输入编号，读取编号	
149-155	遍历玩家集合查找匹配的用户	
157-160	如果用户不存在，提示错误并继续循环	
162-167	如果选择选手登录，校验是否是Gamer类型，是则进入选手菜单	
168-173	如果选择管理员登录，校验是否是MatchAdmin类型，是则进入管理员菜单	
174-176	其他选项：输出无效选项提示	
177	<code>}</code>	while循环结束
178	<code>}</code>	main方法结束
179	<code>}</code>	类定义结束